

Brains – Client Knowledge & Learning Layer

A "brain" is **not a fine-tuned model**. It's a curated profile + a retrievable knowledge base + identity assets + a performance memory, all keyed to one `client_id`. Retrieval beats fine-tuning: update it the moment a client adds an asset, cite real sources, cost cents not training runs – and layer learning on top without retraining anything.

Four layers

1. **Profile** (`client_profiles`) – small, authoritative, always injected. Positioning, audience, banned words, tone, proof points, **outcome definitions** (what a win looks like for this client). Partly seeded from the GAS Sheet (below).
2. **Knowledge base** (`knowledge_chunks`) – large, retrieved on demand, scoped to the

namespace.

3. **Identity** (`clients`) – ElevenLabs

`voice_id`, Higgsfield `soul_id`, HeyGen
`avatar_id`, brand styling, and **sonic
identity** (a consistent music style/finetune
so
a client's videos sound like one body of
work).

4. **Performance memory** (`published_assets` + `performance_metrics`) – what actually worked, fed back into 1 and into the script stage. This is the "learns" in the mission.

Ingestion – two channels, one pipeline

Both converge on `extract → chunk → embed`
`→ store (client_id)`. Run as an Inngest
function

so large syncs don't block a request.

client website

_____ |
GAS Google Sheet → Drive knowledge
folder ─┬→ extract → chunk → embed

```
→ knowledge_chunks
      (manifest)
└
```

A. Client website

Crawl + extract clean, LLM-ready text (e.g. Firecrawl) from the site; chunk; store the page URL in metadata for citation. Good for positioning, voice, public proof points.

B. GAS Google Sheet → Drive folder (the primary, agency-controlled channel)

Your account leads maintain **one Sheet per client** that points to that client's Drive knowledge folder. The Sheet is two things at once:

- **A manifest.** The platform reads the Sheet to discover what's in the folder — file links, types, and per-asset tags (e.g. `case-study`, `tone-sample`, `offer`). It then pulls each referenced file from Drive, parses (PDF/docx/pptx/transcript), chunks, embeds.

On a schedule it re-reads the Sheet + folder

and **re-indexes only what changed**, so the brain stays live as the team drops in new content.

- **A profile-signal source.** Reserve columns/tabs in the Sheet for curated signals — positioning, banned words, target segments, outcome definitions. These flow straight into `client_profiles`, so onboarding a brain is "fill the Sheet, point it at the folder."

Access: connect via the Google Drive + Sheets APIs using a GAS service account with read access to the client folders. Store the Sheet ID on `knowledge_sources` (`type='gsheet'`, `uri=<sheet_id>`); the folder ID is resolved from the Sheet.

Chunking: ~500–800 tokens with overlap; attach `{source_id, title, url, tags}` to every chunk for traceable, tag-filterable citation. Embeddings come from a dedicated embeddings

model (e.g. Voyage) — cheap (cents per 1M tokens); match its output dimensions to the `vector()` column in `knowledge_chunks`. Use **Haiku 4.5** for any LLM work during ingestion (tagging, summarising) — it's high-volume and low-complexity.

Profile builder (onboarding)

The Sheet does most of the work: auto-draft the profile from the Sheet's signal columns + the ingested knowledge, then a human approves (`is_live = true`) before the brain can produce. Never let an unreviewed profile go live.

Retrieval at generation time

For the active brain, the script stage:

1. Loads the live `client_profiles` row in full (rules + outcomes).
2. Embeds the brief; pulls top-k `knowledge_chunks` **filtered by** `client_id` (optionally

by tag, e.g. prefer `case-study` chunks for the "example" beat).

3. Adds **performance context** — the client's best-performing past hooks/structures for the target segment (from the learning layer).
(Iteration 2; skipped in the first build.)
4. Assembles: profile + retrieved facts + winning patterns + style exemplars → generates.

```
select content, metadata
from knowledge_chunks
where client_id = $1
order by embedding <=> $2          --
      cosine distance, hnsw index
limit 8;
```

The learning loop (Iteration 2 — not in the first build)

Deferred to Iteration 2, when the platform goes to market. Iteration 1 is not Meta-integrated; performance feedback and ad-loading come later. The schema seams

exist

now so this lights up without a rewrite.

This is what makes it intelligent rather than just automated.

1. **Capture.** When a finished video is published, log it in `published_assets` (platform, URL, segment, posted_at).
2. **Measure.** Pull performance back in via your existing GAS campaign-analyst skills (Meta/TikTok/X/LinkedIn): retention/watch-through, CTR, engagement, and conversions where tracked. Store in `performance_metrics`, keyed to the asset + segment.
3. **Distil.** Periodically rank a client's assets by the outcome that matters for *them* (their `outcome_definitions`, not vanity metrics — reuse your integrity guardrails). Promote winners into `client_profiles.exemplars`; note losing patterns to avoid.
4. **Apply.** Step 3 of retrieval injects "what's worked for this client/segment" into the

next script. Each brain compounds: GAS
PSI learns PSI's winning angles, Lernalot
learns
which segment hooks convert. Over time
the system isn't guessing — it's pattern-
matching
on its own track record per client.

Isolation test (must pass before brains ship — Phase 2)

Seed two brains with distinct, identifiable facts.
Query brain A for a fact that exists
only in brain B. The correct result is **nothing**.
Automate as a regression test — it
guards the agency's core trust promise (and a
future customer's data).